

CSC 4480 Final Project

Group #8

Liz Bossert, Theo Musilli, Reena Saraiya

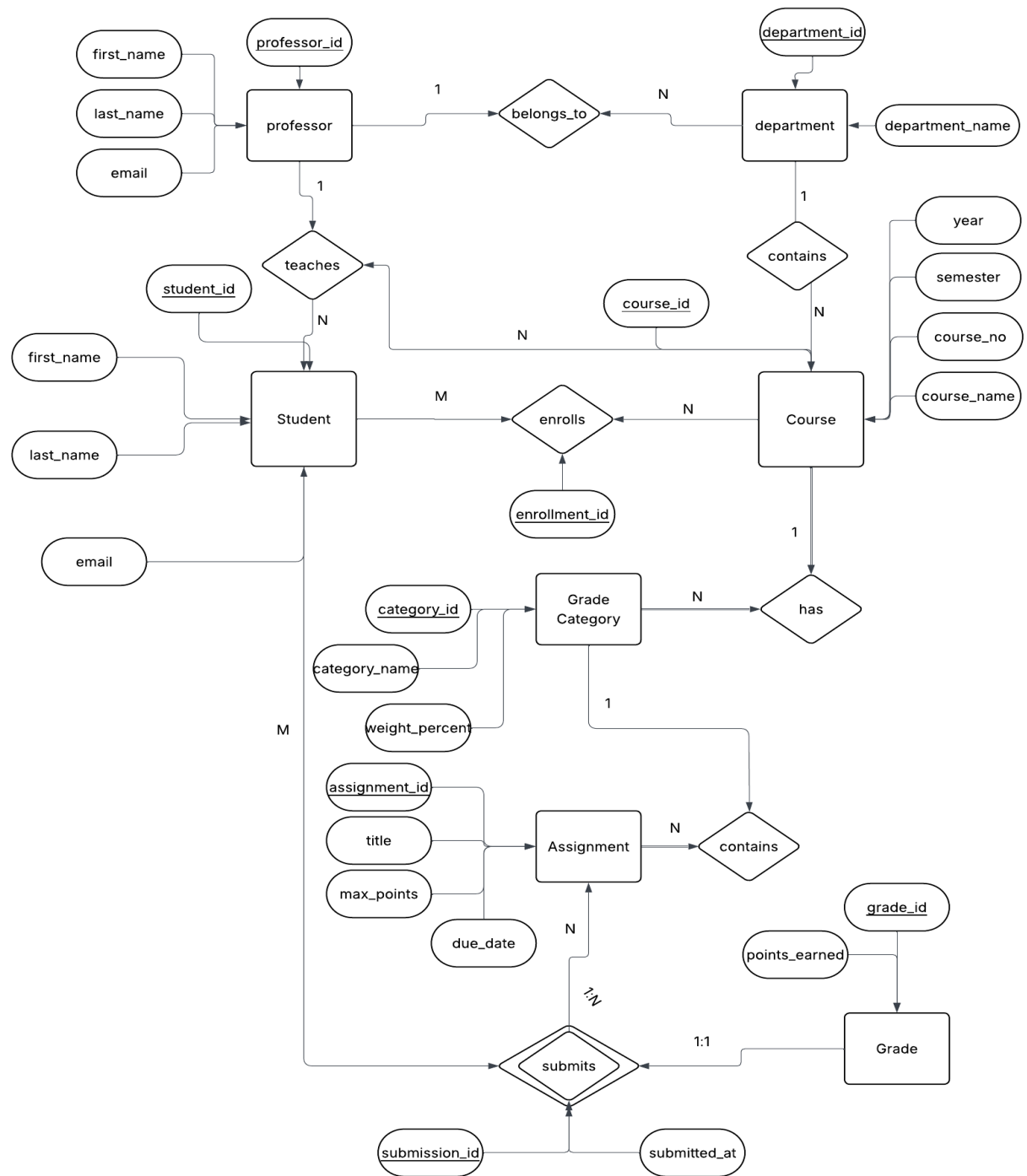
Project Overview

- Relational database for managing course grades
- Tracks:
 - Students, Professors, Courses
 - Assignments, Submissions, Grades
- Supports multi-course, multi-student environment
- Designed for accuracy and data integrity

Database Design

- Strong entities:
 - Student, Professor, Course, Department
- Relationship tables:
 - Enrollment (Student ↔ Course)
 - Submission (Student ↔ Assignment)
- Hierarchical grading structure:
 - Course → Category → Assignment → Grade

Database Design



Key Relationships

- 1:N:
 - Course → Grade Categories
 - Category → Assignments
- M:N:
 - Students ↔ Courses (Enrollment)
 - Students ↔ Assignments (Submission)
- 1:1:
 - Submission → Grade
 - Professor → Department

Constraints

- **Primary Keys**
 - Each table has a unique identifier (ensures entity integrity)
- **Foreign Keys**
 - Enforce relationships between tables (maintains referential integrity)
- **NOT NULL Constraints**
 - Required fields for key relationships (e.g., `student_id`, `course_id`)
- **UNIQUE Constraints**
 - Student email must be unique
 - Prevents duplicate enrollments (*student*, *course*)
 - Prevents duplicate submissions (*assignment*, *student*)
 - Ensures one grade per submission (1:1 relationship)
- **CHECK Constraints**
 - `points_earned` ≥ 0 (valid grades)
 - `max_points` > 0 (valid assignments)
 - `weight_percent` $\in (0, 100]$ (valid category weights)
- **Trigger enforces:**
 - Category weights $\leq 100\%$ per course

Trigger

Grade Category Weight Enforcement Trigger

Ensures total grading weights per course do not exceed 100%

Runs automatically after INSERT, UPDATE, or DELETE on GRADE_CATEGORY

Aggregates all category weights for each course

Rejects any operation that causes invalid total weight

Maintains consistency for final grade calculations

```
--Trigger for grade_category
CREATE OR REPLACE TRIGGER trg_check_category_weight
AFTER INSERT OR UPDATE OR DELETE ON GRADE_CATEGORY
DECLARE
    total_weight NUMBER(6,2);
BEGIN
    FOR rec IN (SELECT DISTINCT course_id FROM GRADE_CATEGORY) LOOP
        SELECT NVL(SUM(weight_percent), 0)
        INTO total_weight
        FROM GRADE_CATEGORY
        WHERE course_id = rec.course_id;

        IF total_weight > 100 THEN
            RAISE_APPLICATION_ERROR(
                -20001,
                'Total weight_percent exceeds 100% for a course'
            );
        END IF;
    END LOOP;
END;
/
```

```
SELECT 'Students Enrolled in Principles of Finance'

SELECT S.student_id, S.first_name, S.last_name, S.em
FROM STUDENT S, ENROLLMENT E, COURSE C
WHERE S.student_id = E.student_id
AND E.course_id = C.course_id
AND UPPER(C.course_name) = UPPER('Principles of Fina
```

```
SELECT 'Student Count Per Course' FROM DUAL;
```

```
SELECT C.course_name, COUNT(E.student_id)
```

Script Output x | Query Result x | Query Result 1 x

SQL | All Rows Fetched: 5 in 0.011 seconds

	STUDENT_ID	FIRST_NAME	LAST_NAME	EMAIL
1	02481234	Theo	Musilli	tmusilli
2	02493429	Liz	Bossert	lbossert
3	02476851	Reena	Sariya	rsariya@
4	02512937	Dave	Ramsey	dramsey@
5	02468315	Charles	Payne	cpayne@v

```
SELECT 'Student Count Per Course' FROM

SELECT C.course_name, COUNT(E.student
FROM COURSE C, ENROLLMENT E
WHERE C.course_id = E.course_id
GROUP BY C.course_name
ORDER BY COUNT(E.student_id) DESC;
```

Script Output x | Query Result x | Query Result 1 x

SQL | All Rows Fetched: 3 in 0.016 seconds

	COURSE_NAME	COUNT(E.STU
1	Portfolio Management	
2	Financial Derivatives	
3	Principles of Finance	

```
SELECT '3 Courses Taught by Professor Buffe

SELECT C.course_name, C.course_no, C.semest
FROM COURSE C, PROFESSOR P
WHERE C.professor_id = P.professor_id
AND UPPER(P.last_name) = UPPER('Buffett');

SELECT 'Students Enrolled in Principles of
```

Script Output x | Query Result x | Query Result 1 x

SQL | All Rows Fetched: 3 in 0.016 seconds

	COURSE_NAME	COURSE_NO	SEMI
1	Principles of Finance	FIN2010	Spring
2	Portfolio Management	FIN3850	Spring
3	Financial Derivatives	FIN4350	Spring

Test Cases

```

SELECT 'Assignments Due After March 15 2026' FROM DUAL;

SELECT A.title, A.due_date, GC.category_name, C.course_name
FROM ASSIGNMENT A, GRADE_CATEGORY GC, COURSE C
WHERE A.category_id = GC.category_id
AND GC.course_id = C.course_id
AND A.due_date > TO_DATE('2026-03-15', 'YYYY-MM-DD')
ORDER BY A.due_date ASC;

SELECT 'Student Lookup by Last Name: Sariya' FROM DUAL;

SELECT * FROM STUDENT

```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x Query Result 4 x

SQL | All Rows Fetched: 7 in 0.011 seconds

	TITLE	DUE_DATE	CATEGORY_NAME	COURSE_NAME
1	Case Study 2 - Stock Valuation	18-MAR-26 00:00:00	Case Studies	Principles of Finance
2	Final Exam	28-APR-26 00:00:00	Final Exam	Principles of Finance
3	HW3 - CAPM Analysis	20-MAR-26 00:00:00	Homework	Portfolio Management
4	Final Exam	29-APR-26 00:00:00	Final Exam	Portfolio Management
5	Case Study 2 - Black-Scholes Model	30-MAR-26 00:00:00	Case Studies	Financial Derivatives
6	Midterm Exam	16-MAR-26 00:00:00	Midterm	Financial Derivatives
7	Final Exam	27-APR-26 00:00:00	Final Exam	Financial Derivatives

```

SELECT 'Grade Category Weight Totals Per Course' FROM DUAL;

SELECT C.course_name, SUM(GC.weight_percent)
FROM GRADE_CATEGORY GC, COURSE C
WHERE GC.course_id = C.course_id
GROUP BY C.course_name
ORDER BY C.course_name ASC;

SELECT 'Assignments Due After March 15 2026' FROM DUAL;

```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x Query Result 4 x

SQL | All Rows Fetched: 3 in 0.004 seconds

	COURSE_NAME	SUM(GC.WEIGHT_PERCENT)
1	Financial Derivatives	100
2	Portfolio Management	100
3	Principles of Finance	100

Test Cases

Final Grade Calculation

```
SELECT 'Final Grade Per Student Per Course' FROM DUAL;

SELECT S.student_id, S.first_name, S.last_name, C.course_name,
       SUM((G.points_earned / A.max_points) * GC.weight_percent) /
       SUM(GC.weight_percent) * 100 AS final_grade
FROM GRADE G, SUBMISSION SB, ASSIGNMENT A, GRADE_CATEGORY GC, COURSE C, ENROLLMENT E, STUDENT S
WHERE G.submission_id = SB.submission_id
AND SB.assignment_id = A.assignment_id
AND A.category_id = GC.category_id
AND GC.course_id = C.course_id
AND SB.student_id = S.student_id
AND E.student_id = S.student_id
AND E.course_id = C.course_id
GROUP BY S.student_id, S.first_name, S.last_name, C.course_name
ORDER BY S.last_name ASC, C.course_name ASC;
```

- **Joins all grading tables together**
 - Student → Enrollment → Course
 - Assignment → Submission → Grade → Category
- **Calculates assignment performance**
 - $\text{points_earned} \div \text{max_points} \rightarrow \text{percentage score per assignment}$
- **Applies category weights**
 - Multiplies each score by its weight_percent
- **Aggregates results**
 - Sums weighted scores across all assignments
 - Groups by student and course
- **Normalizes to final grade**
 - Divides by total weight and scales to 100
- **Handles incomplete records**
 - Only includes assignments that have been graded
 - Ungraded or missing submissions are naturally excluded from the calculation
- **Design choice**
 - Final grade is computed dynamically
 - Avoids inconsistencies and reflects real-time updates

CRU DEMO

- **Create:** Insert student / submission / grade
- **Read:** Query students, courses, grades
- **Update:** Modify grade values

Conclusion

- Fully normalized relational design
- Strong data integrity enforcement
- Supports full gradebook functionality
- Scalable and extensible

Thank you!

Questions?